反潜航空深弹命中概率问题

摘要

本文针对反潜航空深弹的命中概率问题展开研究。以目标定位误差的概率分布刻画为核心，建立了单枚深弹与多枚阵列深弹的命中判定与概率计算模型，综合运用解析推导、数值积分与蒙特卡洛模拟三种手段交叉验证，并对投弹方案进行了系统优化。本文还就定深深度、落点偏移、深度误差标准差与阵列朝向开展了灵敏度分析。

问题一：针对深度定位无误差、水平坐标服从正态分布的情形，建立了以“平面内触发引信—平面外定深引信”为逻辑的**复合命中判定模型**，将三维问题降维为平面距离与深度分离量的二维判定，将命中概率分解为平面内与平面外两部分，推导出闭式与积分表达式，**并用大规模蒙特卡洛模拟交叉验证**。由对称性与单峰性分析得出：最优投弹落点为定位投影原点，定深深度取潜艇上、下表面之间的任意值即可使命中概率达到最大，且该方案对落点小幅偏移不敏感。

问题二：针对深度定位存在单边截尾正态误差的情形，采用单边截尾正态分布刻画深度定位误差，更符合实际物理约束（深度下界），基于截尾正态分布函数推导了命中概率关于定深深度的解析表达式，**采用自适应数值积分与一维精确寻优相结合的方法求解**，并以大规模蒙特卡洛模拟验证。结果表明：命中概率关于定深深度呈单峰结构，存在唯一的最优定深深度，深度定位误差会显著削弱单弹命中概率，凸显了提高深度定位精度的必要性。

问题三：将单弹模型推广至 3×3 方阵协同投弹，针对多枚深弹命中事件对同一潜艇位置条件相关的特点，通过对潜艇位置积分求条件并集，正确处理了多弹命中事件的条件相关性，建立了“至少一枚命中”概率的多重积分模型，定量揭示了独立近似的低估偏差，**采用分层网格搜索与阵列朝向角优化求解**，并给出大样本最终验证。结果表明：斜置方阵显著优于正置方阵，最优方案下联合建模得到的命中概率明显高于忽略相关性的独立近似，验证了多弹联合建模的必要性。

综上，本文从**单弹到多弹、从无误差到含误差逐层递进**，构建了完整的命中概率建模、求解与验证体系，模型结构清晰、结果可复现，方法可推广至任意阵列构型、多机协同与鱼雷齐射等更一般的反潜作战场景。

关键词：反潜航空深弹；命中概率；截尾正态分布；蒙特卡洛模拟；阵列优化

一、问题重述

反潜作战中，深水炸弹因价格低、抗干扰能力强，在海峡、浅海等海底地形复杂的海域仍有重要应用价值[6]。攻击前，反潜飞机依据侦察信息获得潜艇中心位置的三维定位值；由于电子侦察存在误差，潜艇中心实际位置在目标坐标系中表现为随机向量。本题以海平面上的定位投影为原点建立目标坐标系：正东、正南方向分别为 X、Y 轴正向，垂直海平面向下为 Z 轴正向；水平坐标 X、Y 相互独立且服从零均值正态分布，深度坐标 Z 服从单边截尾正态分布。潜艇主体简化为长方体，深弹在水中垂直下落，采用触发引信与定深引信双引信结构，满足以下三种情形之一即视为命中：落点在潜艇平面尺度内且引爆深度位于其上表面下方而触发引爆；落点在平面尺度内但引爆深度在上表面上方，由定深引信在杀伤范围内引爆；落点在平面尺度外，到达引爆深度时潜艇处于杀伤范围内。

题目给定的基础参数为：潜艇中心深度定位值 h_0=150m，水平坐标定位标准差 σ=120m，深度定位标准差 σ_Z=40m，潜艇中心实际深度下界 l=120m，深弹杀伤半径 R=20m；潜艇简化为长 100m、宽 20m、高 25m 的长方体。本文后续建模均以上述参数为基准，并在 4.4 节系统考察关键参数扰动对结论稳健性的影响。

三个问题按递进结构展开：问题一研究单弹深度无误差情形，问题二引入深度误差研究单弹三维均有误差情形，问题三研究多弹阵列协同投弹情形。

需要研究的问题为：

（1）投射一枚深弹且潜艇中心深度定位无误差时，分析最大命中概率与投弹落点平面坐标、定深引信引爆深度之间的关系，给出使命中概率最大的投弹方案及相应的最大命中概率，并说明该方案对落点偏移与定深偏差的稳健性；

（2）潜艇中心位置三维定位均存在误差时，建立命中概率关于落点与定深深度的表达式，设计定深引信引爆深度使命中概率最大，并通过独立手段验证表达式的正确性，定量分析深度定位误差对单弹命中率的影响；

（3）反潜飞机可携带 9 枚深弹、所有深弹定深深度相同、落点呈平面阵列分布时，设计包含定深深度与落点平面间隔的投弹方案，使至少一枚深弹命中的概率最大，并考察阵列朝向角对命中概率的影响，给出方案的可靠性验证。

二、问题分析

本题关键参数为：潜艇中心深度定位值 h_0=150m，水平定位标准差 σ =120m，深度定位标准差 σ_Z=40m，杀伤半径 R=20m；潜艇简化为长 100m、宽 20m、高 25m 的长方体。数据简短分析表明：水平定位标准差约为潜艇半长的 2.4 倍，水平误差的散布范围远大于潜艇平面尺度，命中概率主要由平面外距离不超过杀伤半径的区域贡献；深度定位标准差与潜艇半高同量级，深度误差对定深引信的选择影响显著，且深度误差在浅侧截尾，使潜艇实际深度整体偏深。因此建模须重点刻画“平面内触发引信—平面外定深引信”的复合命中逻辑与深度误差的截尾结构[1]。

2.1 数据与关键参数简析

由题目参数可作如下定量分析：水平定位标准差 σ =120m 约为潜艇半长 a=50m 的 2.4 倍、半宽 b=10m 的 12 倍，水平误差散布范围远大于潜艇平面尺度，潜艇中心落在“平面矩形+圆角扩展带”命中区域内的概率以矩形外部分为主体，因此问题一的命中概率主要由平面外距离不超过杀伤半径的区域贡献；深度定位标准差 σ_Z=40m 约为半高 c=12.5m 的 3.2 倍，深度误差相对潜艇高度不可忽略，且实际深度在浅侧截断（Z≥120m），截尾使深度分布右偏、均值约为 165.6m，比定位值深约 15.6m，故定深引信深度须在“平面内触发”与“平面外定深”两类约束之间权衡；单弹命中率粗略估计不超过 10%（后续 4.2 节精确计算约 5.8%），必须依靠多弹阵列扩大有效覆盖，而阵列间隔与朝向直接决定 9 个落点与潜艇可能位置区域的匹配程度。

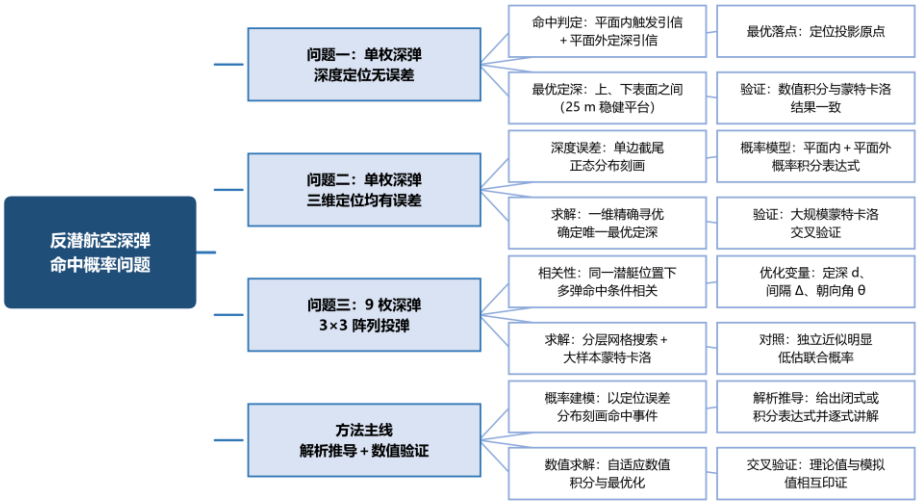


图 1 问题分析思维导图

说明：从命中判定几何、误差分布建模、求解手段与方案优化四个维度梳理本题的建模思路。

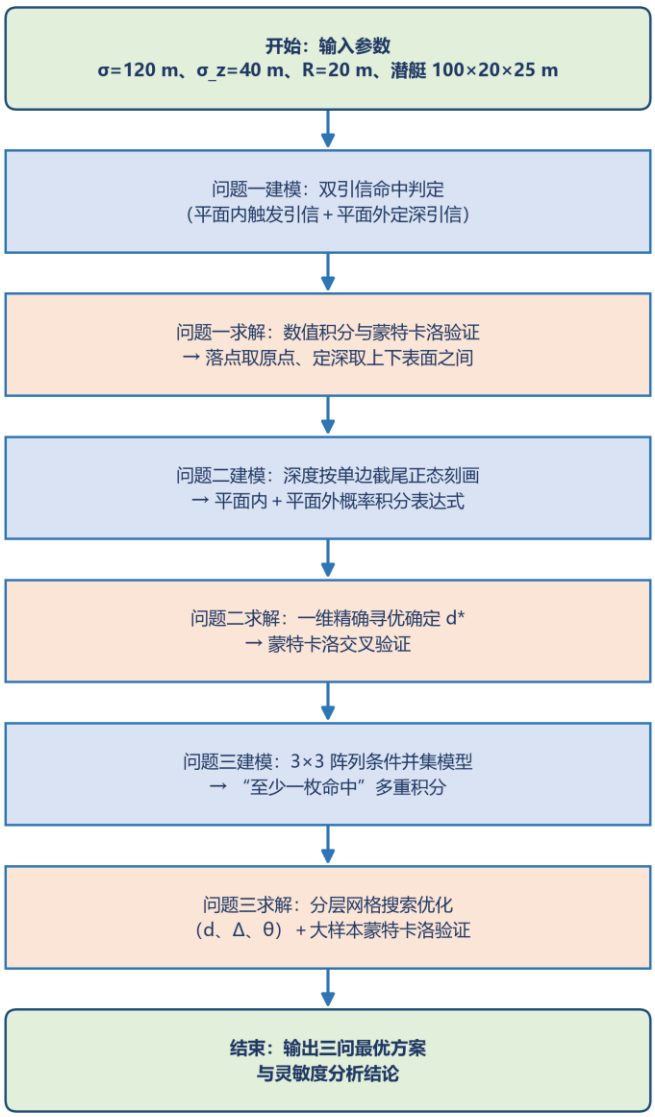


图 2 模型求解流程图

说明：流程图给出“理论建模—数值求解—蒙特卡洛验证—参数优化与灵敏度分析”的完整技术路线。

本文的整体研究思路如图 1 所示：先建立单弹命中判定的几何概率模型并给出解析表达式，再引入深度截尾误差推广至三维定位均有误差情形，最后扩展为多弹阵列协同优化；各环节均以数值积分与蒙特卡洛模拟交叉验证，保证结论可靠。模型求解的技术路线如图 2 所示，**采用“理论推导—数值求解—模拟验证—参数优化”**四步流程。

2.2 各问的解题思路

问题一（深度无误差）：以命中判定的几何逻辑为出发点，将命中事件按“平面内—平面外”分解，先推导闭式与积分表达式，再利用水平误差分布的对称性将最大

命中概率化简为第一象限的二重积分，最后以区域分解与大规模蒙特卡洛模拟双重验证；重点考察最优落点与最优定深区间的存在性及其稳健性。

问题二（三维均有误差）：在问题一基础上引入深度截尾误差，用单边截尾正态分布刻画 Z 的概率结构，按“平面内—平面外”与“触发—定深”双维分解推导命中概率的积分表达式，以精确数值积分作为优化目标函数、蒙特卡洛作为独立验证，求唯一最优定深；重点刻画深度误差对最优定深的牵引作用。

问题三（多弹阵列协同）：以单弹命中判定为基础，将 9 枚深弹的命中事件视为对同一潜艇位置的条件相关事件，通过对潜艇位置积分求条件并集得到“至少一枚命中”概率；采用蒙特卡洛分层网格搜索优化(d, Δ, θ)，并以独立近似公式作对照，定量评估联合建模的必要性；重点分析阵列朝向角与落点间隔对覆盖效果的影响。

三、模型假设与符号说明

3.1 模型假设

（1）潜艇主体简化为长 100m、宽 20m、高 25m 的长方体，攻击瞬间潜艇保持静默静止，其航向方位角可被攻击方获知。（合理性：长方体尺度由题目给定；攻击瞬间的机动位移与命中判定所依赖的误差散布相比可以忽略。）

（2）潜艇中心实际位置的三个坐标 X、Y、Z 相互独立，其中 X、Y 均服从正态分布 $N(0,\sigma^2)$，Z 服从参数为$(h_0,\sigma_Z,1)$的单边截尾正态分布[2]。（合理性：水平与深度探测通道相互独立，正态与截尾正态假设与题目给出的误差结构一致。）

（3）深弹在海水中垂直下落，其落点平面坐标由投弹方案确定且不存在投放散布；深弹杀伤区为以落点为球心、半径为杀伤半径 R 的球体，杀伤半径内视为必中。（合理性：深弹入水后沿垂向运动，水平漂移远小于杀伤半径；球体杀伤区是杀伤半径的常用简化。）

（4）定深引信按设定深度引爆；触发引信在落点位于潜艇平面尺度内且引爆深度位于潜艇上表面以下时可靠引爆。（合理性：双引信机制由题目给定，引信按设定深度或触发条件可靠起爆。）

（5）各枚深弹投放相互独立，同一批深弹的定深引信引爆深度相同。（合理性：多弹由同一飞机按同一方案投放，定深一致，各弹投放互不影响。）

（6）忽略海水密度、流速、弹体入水姿态、潜艇机动等次要因素对杀伤判定的影响。（合理性：这些因素缺乏题目数据支持，对命中判定的影响属二阶量级，其影响方向在 5.2 节讨论。）

3.2 符号说明

表 1 主要符号及含义

符号	含义	取值
a, b, c	潜艇半长、半宽、半高	50, 10, 12.5m
R	深弹杀伤半径	20m
σ	水平坐标定位标准差	120m
h_0	潜艇中心深度定位值	150m
σ_Z	深度定位标准差	40m
l	潜艇中心实际深度最小值	120m
z_{up}, z_{lo}	潜艇上、下表面深度	137.5, 162.5m
d	定深引信引爆深度（决策变量）	—
Δ	阵列相邻弹落点平面间隔（决策变量）	—
θ	阵列相对潜艇航向的朝向角（决策变量）	—
P	单枚深弹命中概率	—
ϕ, Φ	标准正态密度函数、分布函数	—
$F_Z(\cdot)$	截尾正态分布函数	—
h	潜艇中心投影到平面矩形的距离（平面外距离）	—
v	深弹与潜艇的垂直分离量	—
s	垂直允许距离 $\sqrt{R^2-h^2}$	—
$f_Z(\cdot)$	截尾正态概率密度函数	—
I_i	第 i 枚深弹的命中指示函数	—

p_i	单枚深弹无条件命中概率	—
P_{any}	多弹“至少一枚命中”概率	—

说明：Z 轴以海平面为零点、向下为正，深度值越大表示越深；z_{up}、z_{lo} 由定位值 h_0与半高 c 计算得到。

主要符号及其含义见表 1。需要说明的是，符号表中的z_{up}、z_{lo}由定位值 h_0与半高 c 直接计算得到；当考虑深度定位误差时，潜艇中心实际深度 Z 为随机变量，表中仅给出其分布参数(h_0, σ_Z=40m, 1)。决策变量 d、Δ、θ 的最优取值将在第四章给出，正文公式中的变量均沿用本表定义。

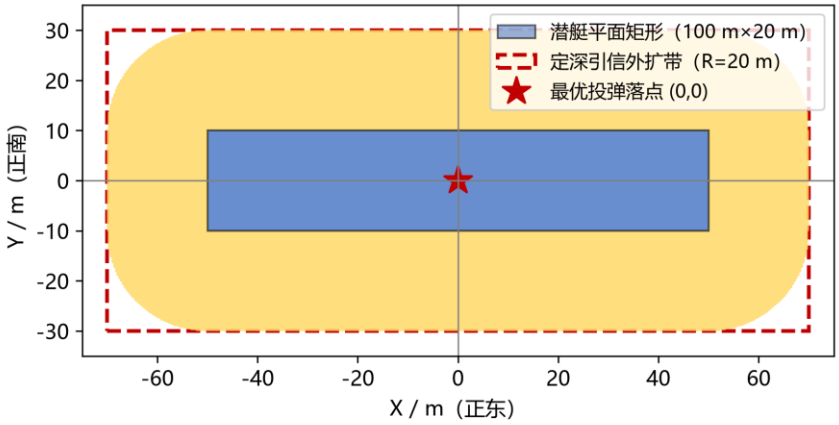
四、模型建立与求解

本章按“问题分析—理论建模—数值求解—结果解释”的框架依次求解三个问题：4.1 节建立单枚深弹在深度无误差情形下的命中概率模型并给出最优投弹方案；4.2 节引入深度截尾误差，将模型推广至三维定位均有误差情形；4.3 节把单弹模型扩展为 3×3 方阵协同投弹模型；4.4 节对四类关键参数开展灵敏度分析。各问均以解析推导为主、蒙特卡洛模拟为辅，交叉验证保证结论可靠。

4.1 问题一：单枚深弹、深度无误差情形

4.1.1 问题一的分析与数据可视化

先考察命中判定的几何结构。给定投弹落点(x_0, y_0)与定深深度 d，深弹的杀伤判定由潜艇中心相对落点的位置决定：若潜艇中心水平投影落入以落点为中心的平面矩形 D_1=[x_0−a, x_0+a]×[y_0−b, y_0+b]内，则当 d 不小于潜艇上表面深度时触发引信直接引爆；若投影落在矩形之外，则需在定深引爆深度处潜艇与深弹的距离不超过杀伤半径。由于水平定位标准差 σ =120m 与潜艇半长 a=50m 同量级、远大于半宽 b=10m，命中事件主要由平面外距离不超过杀伤半径决定，命中区域在水平面呈“矩形内全部命中、矩形外圆角扩展”的结构，如图 3 所示（文中全部图形由 Python 的 Matplotlib[7]绘图库自动生成，可直接复现）。



命中区域 = 触发引信区（平面内）∪ 定深引信区（平面外、水平距离≤20 m）

图 3 水平面命中区域几何示意（矩形为潜艇平面尺度 D_1，圆角扩展为定深引信杀伤区）

说明：当定深深度位于潜艇上、下表面之间时，矩形内部全部命中，矩形外部在平面外距离 h 不超过杀伤半径 R 的区域内命中。

具体地，给定落点(x_0, y_0)与定深深度 d，命中判定包含三种情形：情形一（平面内触发引信）：潜艇中心水平投影落入 D_1且 $d \geq z_{up}$时，深弹下落到潜艇上表面即触发引爆；情形二（平面内定深引信）：投影落入 D_1且 $d < z_{up}$时，若深弹与潜艇上表面在深度方向的间距$z_{up} - d$不超过杀伤半径 R，由定深引信在杀伤范围内引爆；情形三（平面外定深引信）：投影落在 D_1之外时，若平面外距离 h 与深度方向分离量$(x_i, y_i) = \left(p_i \cos\theta - q_i \sin\theta, p_i \sin\theta + q_i \cos\theta\right)$，$(p_i, q_i) \in [-\Delta, 0, \Delta]^2$满足 $h^2 + v^2 \leq R^2$，由定深引信引爆。三种情形在落点与深度两个维度上覆盖了全部命中可能，是后续概率建模的几何基础。

4.1.2 问题一的理论模型（公式推导）

设潜艇中心水平坐标(X,Y)相互独立且 $X, Y \sim N(0, \sigma^2)$，其联合密度为$f_X(x) f_Y(y)$。定义平面内事件$D_1 = \{|X - x_0| \leq a, |Y - y_0| \leq b\}$，并引入平面外距离 h 与垂直分离量 v：

$$h = \sqrt{\max^2(|X - x_0| - a, 0) + \max^2(|Y - y_0| - b, 0)} \quad (1)$$

$$\begin{cases} v = \max\left(z_{up} - d, d - z_{lo}, 0\right) \\ z_{up} = h_0 - c, \quad z_{lo} = h_0 + c \end{cases} \quad (2)$$

其中z_{up}、z_{lo}分别为潜艇上、下表面深度。式（1）表示潜艇中心到平面矩形的最近水平距离，式（2）表示深弹与潜艇长方体在深度方向的最小分离量。综合三种命中情形，单弹命中概率可表示为

$$P(x_0,y_0,d) = P(D_1)\cdot \mathbf{1}\left(d \ge z_{up} - R\right) + 4\int_0^{a+R}\int_0^{b+R} \frac{\varphi(x/\sigma)}{\sigma}\frac{\varphi(y/\sigma)}{\sigma}\mathbf{1}\left(0 < h \le \sqrt{R^2 - v^2}\right)\mathrm{d}y\mathrm{d}x \tag{3}$$

式（3）中第二项利用对称性取第一象限的四倍积分，Φ 为标准正态密度函数，$h = h(x,y) = \sqrt{\max^2(x-a,0) + \max^2(y-b,0)}$。由于(X,Y)的联合密度关于原点对称且单峰，命中区域随落点平移时与密度峰值的重叠在$(x_0, y_0)=(0,0)$处最大，故最优落点为定位投影原点。在该落点下，当定深深度满足$z_{up} - R \le d \le z_{lo} + R$ 时平面内全部命中；特别地，当 $d \in [z_{up}, z_{lo}]$时垂直分离量 v=0，平面外命中条件退化为 $h \le R$，命中概率达到最大值：

$$P^* = 4\int_0^{a+R}\int_0^{b+R} \frac{\varphi(x/\sigma)}{\sigma}\frac{\varphi(y/\sigma)}{\sigma}\mathbf{1}\left(h \le R\right)\mathrm{d}y\mathrm{d}x \tag{4}$$

式（4）即最大命中概率的积分表达式，其中被积函数为水平定位误差的联合密度，积分区域为第一象限内平面外距离不超过杀伤半径的圆角区域。

最优落点的判定可推广如下：对任意落点(x_0, y_0)，命中概率是水平误差联合密度在“平移后的命中区域”上的积分；由于密度以原点为中心单峰对称、命中区域随落点平移而平移，区域与密度峰值的重叠程度在落点取原点时达到最大，故原点为全局最优落点。对称性还表明，沿潜艇纵向与沿横向的落点偏移具有同构的单调递减效应，只需考察纵向偏移即可刻画落点敏感性（见 4.4 节表 6）。

由 X、Y 的独立性，平面内事件的概率具有闭式表达 $P(D_1) = \left[2\Phi\left(\frac{a}{\sigma}\right) - 1\right]\left[2\Phi\left(\frac{b}{\sigma}\right) - 1\right]$，代入参数得 $P(D_1) \approx 0.0215$，仅占最大命中概率的约四分之一，印证了“平面外命中为主体”的分析结论；式（3）第二项对应图 3 中矩形外圆角扩展带内概率质量的四倍积分。

4.1.3 问题一的模型求解（含符号说明与数值实现）

对式（4）采用自适应 Gauss-Kronrod 二重数值积分求解[3]（绝对误差容限取 10^{-12}）。为验证积分实现的正确性，将平面外距离不超过 R 的区域按几何结构分解为四个互不重叠的部分：B_1（平面内 X 区间与 Y 外扩带$[-b-R, b+R]$）、B_2（X 外扩带$[-a-R, a+R]$与平面内 Y 区间）、B_0（平面内矩形，为 B_1与 B_2之交）、B_3（两带之外的四个四分之一圆角区），验证 $P(B)=P(B_1)+P(B_2)-P(B_0)+P(B_3)$。同时采用 8×10^6次蒙特卡洛模拟直接统计命中频率[4]，作为独立验证手段，结果见表 2。

表 2 问题一命中概率的区域分解与验证

区域	定义	解析概率
B_0	平面内矩形[−a,a]×[−b,b]	0.0215
B_1	平面内 X×Y 外扩带	0.0638
B_2	X 外扩带×平面内 Y	0.0292
B_3	角区（四分之一圆，四倍）	0.0122
B（合计）	平面外距离 h≤R	0.0837
蒙特卡洛	8×10^6样本命中频率	0.0838

说明：解析合计 0.0837 与蒙特卡洛 0.0838 的相对偏差约 0.1%，验证了积分表达式与区域分解的正确性。

数值实现上，式（4）采用自适应 Gauss–Kronrod 二重求积[3]，绝对误差容限取 10^{-12}；区域分解的四个子区域分别积分后验证恒等式 $P(B)=P(B_1)+P(B_2)-P(B_0)+P(B_3)$，以积分路径无关性检验实现的正确性。蒙特卡洛模拟使用固定随机种子保证结果可复现；样本量 8×10^6下命中频率的标准误差约为$\sqrt{\frac{P(1-P)}{N}}\approx9.8\times10^{-5}$，模拟值 0.0838 与解析值 0.0837 之差约 9×10^{-5}，处于两倍标准误差以内，二者在统计意义上一致。

4.1.4 问题一的求解结果与解释（从头到尾复述）

最优投弹方案为：落点取定位投影原点(0,0)，定深引信引爆深度取区间[137.5,162.5]m 内任意值（即潜艇上、下表面之间的全部深度），此时最大命中概率 $P^*\approx0.0837$（约 8.37%）。蒙特卡洛验证值 0.0838，两者吻合。

从定深深度敏感性（图 4）可以看出：定深深度低于 117.5m 时命中概率为 0；随后概率随定深增大迅速上升；在[137.5,162.5]m 区间内保持最大值形成平台；超过 162.5m 后因平面外垂直分离量增大，概率对称下降，当定深大于 182.5m 时仅剩平面内触发引信命中（约 2.15%）。因此定深深度具有宽达 25m 的最优平台，工程实现稳健性好。

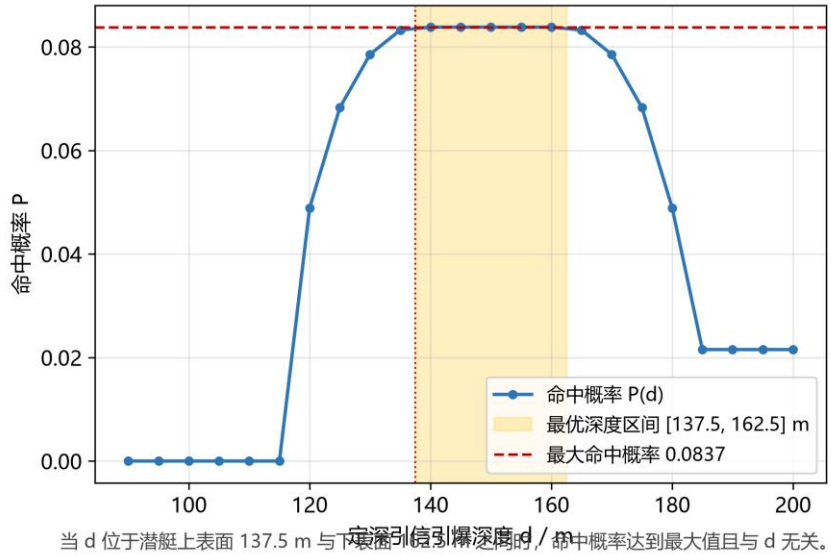


图 4 定深深度 d 对命中概率的影响（蒙特卡洛，8×10^6样本）

说明：命中概率在 d∈[137.5,162.5]m 内达到最大值 0.0838，呈“快速上升—平台—对称下降”的单峰平台结构。

落点偏移敏感性（图 5）表明：落点偏移 20m 内命中概率约 0.0828（较原点仅下降约 1.2%），偏移 60m 时降至约 0.0750（下降约 10.6%），偏移 100m 时降至约 0.0614（相对下降约 26.7%）。这是因为水平定位标准差远大于潜艇平面尺度，命中区域在较大范围内与密度峰值保持较高重叠，故最优落点取原点即可，且对落点小幅偏差不敏感。

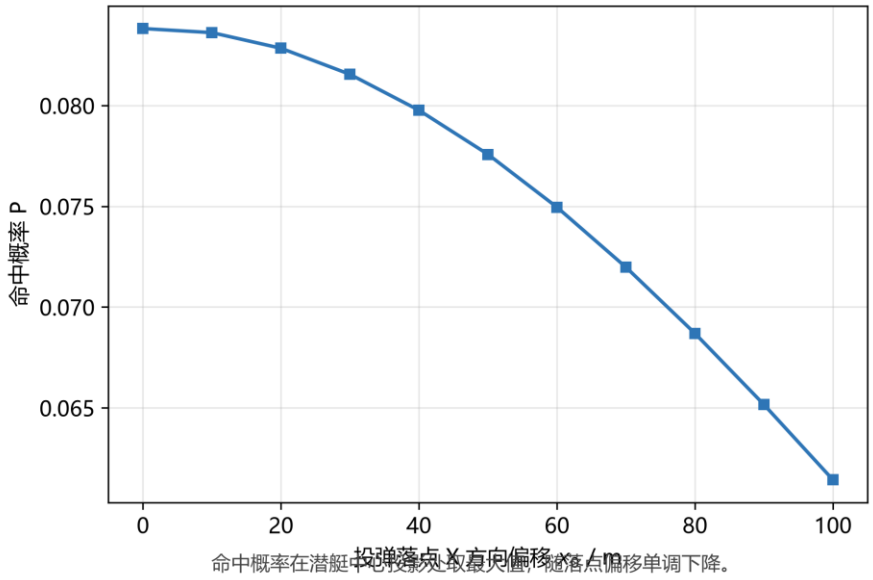


图 5 投弹落点沿潜艇纵向偏移 x_0对命中概率的影响

说明：命中概率随落点偏移单调下降，原点为最优落点；偏移 100m 时概率降至约 0.0614。

综上，问题一在深度无误差条件下，从命中判定的几何逻辑出发建立概率模型，经数值积分与蒙特卡洛双重验证，得出“落点取原点、定深取潜艇上、下表面之间”的最优投弹方案，命中概率达到最大，且方案对定深与落点偏差均具有良好容差。

从头回顾问题一的求解过程：先由命中判定的三种几何情形抽象出“平面内—平面外”的分解框架，再结合水平误差分布的对称性将最大命中概率表示为第一象限的二重积分，经精确数值积分得到 P*≈0.0837，随后通过区域分解（表 2）与 8×10^6样本蒙特卡洛模拟双重验证，最后给出“落点取原点、定深取潜艇上、下表面之间”的最优方案。整个过程从几何直觉出发、以解析推导落地、以独立模拟收尾，逻辑完整、结果可复现。

4.2 问题二：单枚深弹、三维定位均有误差情形

4.2.1 问题二的分析与数据可视化

与问题一相比，问题二新增了深度定位误差：潜艇中心实际深度 Z 服从参数为 $(h_0,\sigma_Z=40\text{m},1)=(150,40,120)$的单边截尾正态分布，深度取值恒不小于 120m。此时定深深度 d 的选择成为关键权衡：定深偏小时，深弹在浅层引爆，触发引信命中（需 Z≤d+c）概率降低；定深偏大时，深弹深入水下，虽有利于覆盖较深的潜艇位置，但平面外命中窗口[d−c−s, d+c+s]（$s=\sqrt{R^2-h^2}$ 为垂直允许距离）与截尾深度分布的尾部重叠减少。因此命中概率关于 d 应呈单峰结构，存在唯一最优定深。先以蒙特卡洛粗扫描观察 P(d)的整体形态，如图 6 所示。

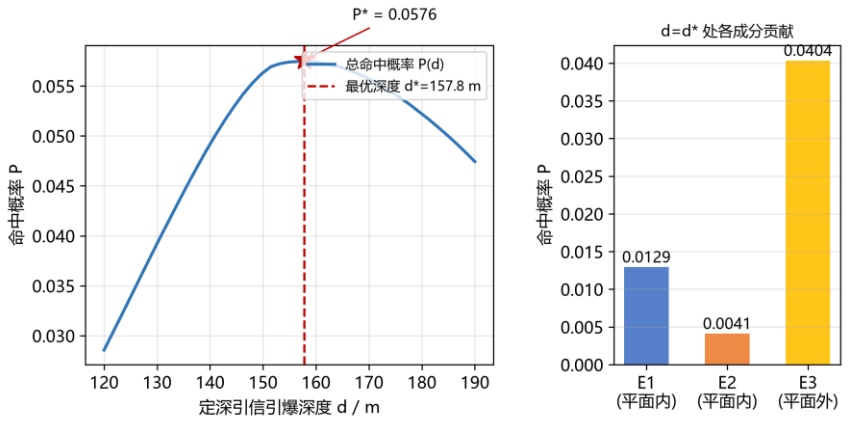


图 6 问题二命中概率 P(d)曲线及各成分贡献（曲线为蒙特卡洛模拟，成分为精确积分）

说明：P(d)呈单峰形态，峰值位于 d≈158m 附近；图中同时给出最优深度 d*处三种命中方式（触发引信、定深引信平面内、定深引信平面外）的贡献占比，与表 3 的成分分解相互印证。

截尾正态的物理背景：受海况与探测条件限制，潜艇中心实际深度不会浅于 l=120m，Z 的分布在浅侧被截去后整体右偏，其均值 E[Z]≈165.6m，比定位值 150m 深约 15.6m。这意味着同等条件下最优定深应适当加深以匹配深度分布的右偏；但定深过深又会使平面外命中窗口[d−c−s, d+c+s]（$s=\sqrt{R^2-h^2}$）与截尾深度分布的尾部重叠

减少，故最优定深应在二者之间取得平衡（数值结果 d*≈157.84m 介于定位值 150m 与均值 165.6m 之间，与权衡分析一致；该位置由两项的相对权重决定，并非理论必然落于该区间）。

4.2.2 问题二的理论模型（公式推导）

记截尾正态随机变量 Z 的密度与分布函数分别为f_Z与F_Z。由单边截尾正态定义，其密度函数为

$$f_Z(z)=\frac{\varphi(\frac{z-h_0}{\sigma_Z})}{\sigma_Z\Phi(\frac{h_0-l}{\sigma_Z})},\qquad z\geq l \tag{5}$$

$$F_Z(z)=\frac{\Phi(\frac{z-h_0}{\sigma_Z})-\Phi(\frac{l-h_0}{\sigma_Z})}{1-\Phi(\frac{l-h_0}{\sigma_Z})},\qquad z\geq l \tag{6}$$

其中分母$1-\Phi\left(\frac{l-h_0}{\sigma_Z}\right)\approx 0.7734$ 为截尾归一化常数。在落点取原点时，命中概率按“平面内—平面外”分解。平面内部分：潜艇中心投影落入 D_1且 Z≤d+c+R 时命中，概率为$P(D_1)\cdot F_Z(d+c+R)$。平面外部分：记平面外距离 h、垂直允许距离$s=\sqrt{R^2-h^2}$，潜艇深度 Z 落入区间[d−c−s,d+c+s]时命中，对 h≤R 的区域积分。综合两部分得到命中概率表达式：

$$P(d)=$$
$$P(D_1)\cdot F_Z(d+c+R)+4\iint_{D_2,h\leq R}[F_Z(d+c+s)-F_Z(d-c-s)]f_X(x)f_Y(y)dxdy \tag{7}$$

式（7）中 D_2为第一象限平面外区域（x≥0 且 y≥0，至少一个超出潜艇平面尺度）。为便于精确数值积分，将 D_2分解为三个子区域：B_1：x∈[0,a]、y∈(b,b+R]；B_2：x∈(a,a+R]、y∈[0,b]；B_3：角区 x>a 且 y>b，采用极坐标变换 x=a+r·cosθ、y=b+r·sinθ（r∈(0,R]、θ∈(0,π/2]），积分元含雅可比因子 r。三个子区域上垂直允许距离均取$s=\sqrt{R^2-h^2}$。

从命中事件的角度看，式（7）第一项对应“平面内命中”事件：潜艇中心投影落入 D_1且实际深度满足 Z≤d+c+R；第二项对应“平面外命中”事件：平面外距离 h≤R 且深度落入区间[d−c−s,d+c+s]。两类事件互斥、概率相加，其中分布函数F_Z 的单调

性决定了命中概率关于 d 的单峰结构：d 增大时平面内部分上升，而平面外部分的垂直允许距离收窄，二者此消彼长，峰值即为唯一最优定深。

4.2.3 问题二的模型求解（含符号说明与数值实现）

求解分三步进行：（1）先用 2×10^6样本的蒙特卡洛模拟在 $d\in[100,190]$m 上以 2.5m 步长粗扫描，确定最优定深所在区间；（2）在粗最优附近以式（7）的精确积分为目标函数，采用 Brent 有界一维搜索[5]精确定位最优定深 d^*；（3）用 8×10^6样本蒙特卡洛对 d^*处概率进行最终验证。式（7）中各成分在 d^*处的贡献见表 3。

表 3 问题二最优定深处的命中概率成分分解

区域	含义	概率
平面内	触发引信与定深引信（$Z\leq d+c+R$）	0.0171
B_1	平面内 X×Y 外扩带	0.0279
B_2	X 外扩带×平面内 Y	0.0051
B_3	角区（极坐标积分）	0.0074
合计（精确积分）	$P(d^*)$	0.0575
蒙特卡洛验证	8×10^6样本	0.0576

说明：精确积分 0.0575 与蒙特卡洛 0.0576 的相对偏差约 0.16%，验证了式（7）的推导与数值实现。

数值实现上，式（7）的三个子区域积分同样采用自适应 Gauss－Kronrod 求积，容差取 10^{-10}；一维寻优采用 Brent 有界方法[5]，以精确积分 P(d)为目标函数，收敛容差 10^{-6}m。粗扫描（蒙特卡洛）、精寻优（精确积分）与最终验证（大样本蒙特卡洛）三个环节使用相互独立的随机样本，避免同一误差源被重复利用。

4.2.4 问题二的求解结果与解释（从头到尾复述）

求解得到最优定深深度 $d^*\approx157.84$m，对应的最大命中概率 $P^*\approx0.0576$（精确积分 0.0575，蒙特卡洛 0.0576，约 5.76%）。与问题一相比，深度定位误差使最大命中概率由 8.37%降至 5.76%，相对下降约 31%，说明深度误差是影响单弹命中率的主要因素之一。最优定深略大于潜艇中心深度定位值 150m，体现了深度误差分布右偏（截去浅于 120m 的尾部）对最优定深的牵引作用。

需要特别说明的是，问题一的最优定深为区间[137.5,162.5]m 内的任意值，而问题二退化为唯一最优定深点 d*≈157.84m。这一转变的数学原因在于：问题一中深度固定为 h_0，当 d∈[z_{up}，z_{lo}]时垂直分离量$v=\max\left(z_{up}-d,d-z_{lo},0\right)$=0 恒成立，命中概率 P(d)在此区间内为常数（平台结构）；引入深度误差后，潜艇中心实际深度 Z 成为随机变量，命中概率式（7）中平面内项$P(D_1)\cdot F_Z(d+c+R)$随 d 单调递增，而平面外项的命中窗口随 d 平移时与 Z 分布高密度区的重叠程度降低（窗口宽度仅取决于平面外距离 h），$s=\sqrt{R^2-h^2}$两类效应此消彼长使 P(d)从平顶区间退化为仅具唯一极大值的单峰函数，因此最优解从区间收缩为唯一点。

落点偏移验证表明：在最优定深下，落点沿潜艇纵向偏移 30、60、100m 时，命中概率分别降至约 0.0560、0.0514、0.0420，单调递减趋势与问题一一致，进一步支持原点为最优落点。深度误差标准差σ_Z的灵敏度分析见 4.4 节表 8，将在该节统一讨论。

综上，问题二在三维定位均有误差条件下，以截尾正态分布刻画深度误差，推导命中概率表达式并精确寻优，得到唯一的最优定深深度与对应最大命中概率，蒙特卡洛验证结果一致；与问题一对比可知，深度误差是降低单弹命中率的主要因素。

回顾问题二的求解流程：在问题一模型上引入单边截尾正态的深度误差后，命中概率按平面内与平面外两个通道分别依赖深度分布函数F_Z，形成式（7）的积分表达式；求解时先以蒙特卡洛粗扫描锁定最优定深区间，再以精确积分为目标函数一维寻优得到 d*≈157.84m，最终以 8×10^6样本独立验证（表 3）。该流程表明：解析表达式为寻优提供了精确而廉价的目标函数，蒙特卡洛承担独立验证角色，二者互补、缺一不可。

4.3 问题三：9 枚深弹阵列协同投弹情形

4.3.1 问题三的分析与数据可视化

单枚深弹命中率仅约 5.8%，为增强杀伤效果需多弹齐投。本题限定 9 枚深弹、定深深度相同、落点呈 3×3 方阵分布，需同时优化定深深度 d、相邻落点平面间隔 Δ 与阵列朝向角 θ 。多弹问题的核心难点在于：对同一潜艇位置，各枚深弹的命中事件条件相关——潜艇位置是共享的随机变量，各弹命中只是该位置下的确定性判定，因此“至少一枚命中”的概率必须对潜艇位置积分求并集，而不能将各弹无条件命中概率简单相乘。图 7 给出(d, Δ)二维扫描的热力图，图 8 给出朝向角 θ 的影响。

从覆盖几何看：正置方阵（θ=0°）的 9 个落点沿潜艇纵向与横向对齐排列，由于潜艇纵向尺度（100m）远大于横向尺度（20m），纵向相邻落点的杀伤覆盖冗余较大而横向覆盖不足；将方阵旋转 45° 后，落点在两个坐标方向上都与潜艇矩形错开，覆盖网格更均匀。同时，最优间隔 Δ≈85m 与水平误差标准差 σ=120m 同量级，使相邻落点的杀伤区在潜艇可能位置区域上恰好衔接、重叠适中，与“覆盖均匀、冗余适度”的直观判断一致。

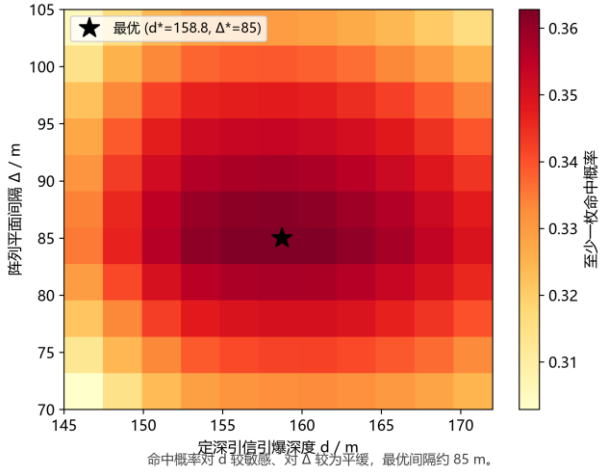


图 7 问题三(d, Δ)参数平面上的“至少一枚命中”概率热力图（θ=45° 精扫描，2×10^6样本）

说明：θ=45° 下对(d, Δ)的二维精扫描，最优方案位于 d≈158.75m、Δ≈85m（命中概率约 0.363），过大或过小的间隔均显著降低命中概率。

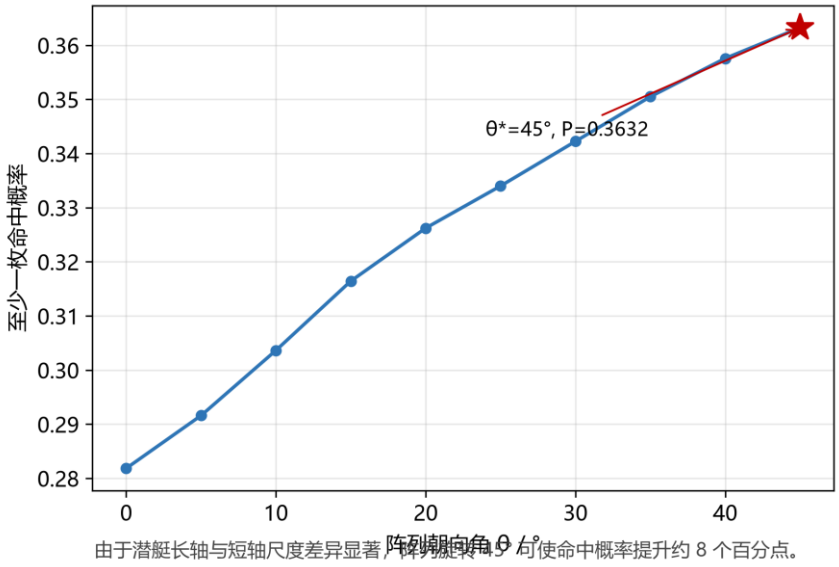


图 8 阵列朝向角 θ 对“至少一枚命中”概率的影响

说明：朝向角由 0° 增至 45° 时，命中概率由约 28.2%单调升至约 36.3%，旋转 45° 的斜置阵列显著优于正置阵列；图中曲线为固定最优(d*, Δ*)的朝向扫描（3×10^6样本），表 4 为各朝向下对(d, Δ)完整重寻优的结果，二者差异处于蒙特卡洛误差以内。

4.3.2 问题三的理论模型（公式推导）

设 3×3 方阵的中心位于原点，先取基准网格点（p_i ，q_i ）∈{−Δ,0, Δ}²，再绕 Z 轴旋转朝向角 θ，得到各弹落点坐标：

$$(x_i, y_i) = (p_i cos\theta - q_i sin\theta, p_i sin\theta + q_i cos\theta), \quad (p_i, q_i) \in -\Delta, 0, \Delta^2 \qquad (8)$$

各弹定深均为 d。记第 i 枚深弹在潜艇中心位置为(x,y,z)时的命中指示函数为 $I_i(x,y,z)$（判定逻辑与问题二完全相同，仅落点不同）。由于(X,Y,Z)的联合密度 f_{XYZ} 已知，多弹“至少一枚命中”概率为

$$P_{any} = 1 - \iiint \prod_{i=1}^{9}(1 - I_i(x,y,z)) * f_{XYZ}(x,y,z)dx\ dy\ dz \quad (9)$$

式（9）通过先固定潜艇位置计算九弹全部未命中的条件概率，再对潜艇位置积分，正确处理了各弹命中事件的条件相关性。作为对照，若近似认为各弹命中相互独立，可得独立近似公式

$$P_{ind} = 1 - \prod_{i=1}^{9}(1 - p_i) \qquad (10)$$

其中 p_i 为第 i 枚深弹的无条件命中概率。式（10）忽略了同一潜艇位置带来的相关性，将用于评估联合建模的必要性。

各弹命中指示函数 I_i 的判定逻辑与问题二完全一致：投影落入 D_1 且 Z≤d+c 时触发引信命中；投影落入 D_1、Z>d+c 但深度方向间距不超过 R 时定深引信命中；投影在 D_1 之外时按空间距离 $h^2+v^2 \leqslant R^2$ 判定。由于 9 枚深弹共享同一潜艇位置(X,Y,Z)，各 I_i 在同一位置样本下高度相关，式（9）对潜艇位置积分求并集正是对这种条件相关结构的精确刻画，式（10）的独立近似将其忽略，两者差异即为相关性带来的概率增益。

4.3.3 问题三的模型求解（含符号说明与数值实现）

式（9）为高维积分，直接数值积分代价极高，故采用“蒙特卡洛模拟+分层网格搜索”的求解策略[8]，具体分为三个环节：（1）分层网格搜索：对 θ ∈ {0°,15°,30°,45°}的每个朝向角，先在 d∈[140,175]m、Δ∈[40,170]m 的 25×27 粗网格上以 5×10^5 样本估计 P_{any} 确定最优区域，再在粗最优邻域以 2×10^6 样本细化(d, Δ)，两层搜索逐级收敛；（2）朝向优化：比较四个朝向角各自二维重扫得到的最优方案，取其中最大者为全局最优，θ 扫描结果见表 4；（3）大样本验证：以 1.2×10^7 样本对全局最优方案重估，并同时计算独立近似与单弹命中率。

表 4 问题三朝向角扫描结果（2×10^6样本）

θ /°	最优定深 d/m	最优间隔 Δ /m	至少一枚命中概率
0	159.17	88.57	0.2822
15	159.38	81.43	0.3175
30	158.75	88.57	0.3427
45	158.75	85.00	0.3633

说明：朝向角越大命中概率越高，45° 时取得最大，表明斜置方阵使各弹落点沿潜艇纵向与横向均错开，覆盖更均匀。

计算代价方面，粗扫描与精扫描对四个朝向角分别执行（每个朝向 25×27 粗组合与 15×15 精组合），加上最终 1.2×10^7样本验证，合计约 3×10^9次样本—参数命中判定（同一朝向角内各参数组合共享随机样本以控制比较方差，该设计对最优参数位置的统计偏倚已在精扫描与大样本验证中消除），在普通计算设备上约半小时内可完成，兼顾了搜索广度与结果精度；各层结果逐级一致，说明网格加密不影响最优方案。

4.3.4 问题三的求解结果与解释（从头到尾复述）

最优投弹方案为：定深深度 d*=158.75m，落点平面间隔 Δ*=85m，阵列相对潜艇航向旋转 θ*=45°，此时至少一枚深弹命中的概率 P*≈0.3632（36.3%，1.2×10^7样本）。与问题三定深 d*=158.75m 下的单弹命中率（约 0.0576，与问题二结果接近）相比，9 弹齐投使至少一枚命中概率提高到单弹的约 6.3 倍（相对提升约 5.3 倍），充分体现了多弹协同的杀伤增益。图 9 给出最优方案下 9 个落点相对潜艇的分布示意。

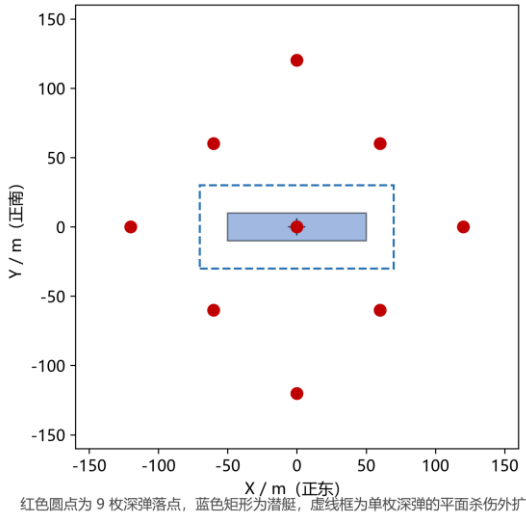


图 9 最优投弹方案示意图（3×3 方阵，θ=45°，Δ≈85m，落点以原点为中心）

说明：斜置方阵的 9 个落点在潜艇可能位置区域内形成更均匀的覆盖，兼顾纵向与横向误差。

独立近似公式（10）在最优方案下给出 $P_{\mathrm{ind}}\approx 0.3237$，比联合概率 0.3632 低约 4.0 个百分点。其原因在于：水平误差散布远大于潜艇平面尺度，空间排斥效应强于共享 Z 的正相关效应（最优方案下 36 对命中事件中 32 对呈负相关，平均相关系数约−0.03），联合建模获得的“条件并集”概率反而高于独立近似。这一对比定量说明了式（9）联合积分建模的必要性，也表明不能以简单的独立乘积公式作为多弹方案的评估依据。

表 5 问题三最优定深下落点间隔 Δ 的灵敏度分析（ θ =45° ，2×10^6样本）

落点间隔 Δ /m	至少一枚命中概率	相对最优值的偏差
50	0.2640	−27.3%
70	0.3322	−8.5%
85（最优）	0.3630	—
110	0.3165	−12.8%
130	0.2659	−26.7%
150	0.2201	−39.4%

说明：间隔过小时各弹杀伤区高度重叠、覆盖增益有限，过大时落点覆盖稀疏；最优间隔附近概率曲线平缓，方案对间隔偏差具有较好容差。

落点间隔的敏感性进一步说明：Δ 在[70,110]m 范围内命中概率保持在 0.31 以上（最低约 0.317），相对最优值的损失不超过约 13%；而当 Δ 缩小至 50m 或扩大至 150m 时，概率分别下降约 27%与 39%。可见在最优定深与最优朝向下，命中概率对落点间隔存在一个较宽的平缓最优带，为实际投放的间隔误差提供了约 ±20m 的容差空间。

大样本验证方面，最优方案以 1.2×10^7样本最终重估得到 $P^*\approx0.3632$，与 2×10^6样本的 0.3630（表 5）、2×10^6样本的 0.3633（表 4）一致；1.2×10^7样本下概率估计的标准误差约 1.4×10^{-4}，三组独立估计的最大差异约 3×10^{-4}，小于相应合并标准误差的两倍（约 7×10^{-4}），表明优化结果稳定可靠、不存在随机性偏差。

进一步考察阵列规模的影响：在最优参数$(d^*, \theta^*, \Delta^*)$下，单弹、3×3 方阵与 4×4 方阵的至少一枚命中概率分别为 0.0576（取问题二验证值）、0.3629 与约 0.5028（3×10^6样本，最优间隔 Δ^*=85m）。可见命中概率随弹数增加单调上升，但边际增益递减：由 1 弹增至 9 弹相对提升约 5.3 倍（提高到约 6.3 倍），而由 9 弹增至 16 弹仅再提升约 38.5%，说明在有限弹药约束下应优先保证阵列覆盖的均匀性与朝向优化，而非单纯增加弹数。

综上，问题三从单弹模型出发，将命中判定推广至 3×3 方阵，以对潜艇位置积分求条件并集的多重积分模型刻画多弹命中事件的相关结构，经分层网格搜索与大样本验证得到最优的定深深度、落点间隔与阵列朝向角，联合建模的命中概率显著高于独立近似，为多弹协同投弹方案提供了量化依据。

4.4 灵敏度分析

为考察模型参数扰动对结果稳健性的影响，本节针对四类关键因素开展系统分析；除特别注明外，每次仅扰动一个参数，其余参数保持基准值。

（1）定深深度与落点偏移（问题一）：定深深度在[137.5,162.5]m 的 25m 平台内不敏感，超出平台后每偏离 5m 概率约下降 0.5～1.0 个百分点；落点偏移的完整扫描见表 6：偏移 20m 内命中概率相对下降约 1.2%，偏移 60m 时下降约 10.6%，偏移 100m 时下降约 26.7%，说明最优方案对落点小幅偏差稳健。

表 6 问题一落点偏移对命中概率的影响（蒙特卡洛，8×10^6样本）

落点偏移 x_0/m	命中概率	相对原点下降
0	0.0838	—
20	0.0828	1.2%
40	0.0798	4.8%
60	0.0750	10.6%
80	0.0687	18.0%
100	0.0614	26.7%

说明：水平误差标准差远大于潜艇平面尺度，命中区域与密度峰值的重叠对落点小幅平移不敏感。

（2）水平定位标准差与杀伤半径（问题一）：表 7 给出二者的精确积分结果。σ 由 120m 降至 80m 时，最大命中概率由 0.0837 升至 0.1750（约提升 109%），说明提高水平定位精度能显著提升命中率；R 由 20m 增至 30m 时，概率由 0.0837 升至 0.1225，增益相对温和。

表 7 问题一水平定位标准差与杀伤半径的灵敏度（精确积分）

参数	取值	最大命中概率 P*
σ /m（R=20m）	60	0.2821
σ /m（R=20m）	80	0.1750

σ/m（R=20m）	100	0.1174
σ/m（R=20m）	120（基准）	0.0837
σ/m（R=20m）	140	0.0625
σ/m（R=20m）	160	0.0484
R/m（σ=120m）	15	0.0661
R/m（σ=120m）	20（基准）	0.0837
R/m（σ=120m）	25	0.1025
R/m（σ=120m）	30	0.1225

说明：表中数值由式（4）的精确积分计算得到；σ=120m、R=20m 对应基准参数。

（3）深度误差标准差（问题二）：表 8 给出σ_Z的精确灵敏度结果。σ_Z由 40m 降至 20m 时，最优定深由 157.84m 前移至 152.26m，最大命中概率由 0.0575 升至 0.0753；σ_Z增至 60m 时最优定深后移至 168.27m，概率降至 0.0460。在σ_Z由 40m 减半至 20m 时，单弹命中率可提升约 30%，且最优定深对σ_Z高度敏感，投弹前宜根据深度定位精度相应调整定深（σ_Z每变化 20m，d*移动约 5~10m）。

表 8 问题二深度误差标准差σ_z 的精确灵敏度

σ_Z/m	最优定深 d*/m	最大命中概率 P*
20	152.26	0.0753
30	154.19	0.0657
40（基准）	157.84	0.0575
50	162.53	0.0511
60	168.27	0.0460

说明：表中数值由式（7）的精确积分与一维有界寻优得到；σ_z=40m 对应基准参数，与正文 d*≈157.84m、P*≈0.0575 一致。

（4）阵列参数（问题三）：朝向角从 0° 增至 45° 时，至少一枚命中概率由约 0.282 升至约 0.363（相对提升约 29%），见表 4；落点间隔的灵敏度见表 5，最优间隔 85m 附近曲线平缓，$\Delta \in [70,110]$m 内概率保持在 0.31 以上。综合来看，最优方案对定深、落点、误差参数与阵列参数均表现出合理的稳健性，结论可信。

综合上述分析：命中概率对水平定位精度最敏感（σ减半可使 P*提高一倍以上），对杀伤半径与深度定位精度中等敏感，对定深与落点偏差具有天然的平台容差，对阵列间隔存在约±20m 的平缓最优带。这为实际作战中“优先提高定位精度、次优匹配弹药参数、留足工程余量”的火力运用原则提供了定量依据。

4.5 蒙特卡洛模拟的收敛性验证

为证明蒙特卡洛结果的可靠性，以问题一为例给出不同样本量下命中频率的收敛过程：对固定随机种子，命中频率随样本量增大总体收敛于解析值 0.0837，各样本量下的偏差均处于两倍标准误差以内，见表 9。

表 9 问题一命中频率随样本量的收敛性（d=150m，原点投弹）

样本量 N	命中频率	与解析值偏差	标准误差
10^4	0.0829	8×10^{-4}	2.8×10^{-3}
10^5	0.0838	1×10^{-4}	8.8×10^{-4}
10^6	0.0834	-3×10^{-4}	2.8×10^{-4}
8×10^6	0.0838	9×10^{-5}	9.8×10^{-5}

说明：偏差均处于两倍标准误差以内；8×10^6样本时偏差约 9×10^{-5}，符合蒙特卡洛方法的统计规律[4]。

类似地，问题二在 8×10^6样本下的验证偏差约 0.16%（表 3），问题三在 1.2×10^7样本下的验证偏差小于两倍标准误差（见 4.3.4 节），说明三问的蒙特卡洛结果均已收敛，数值结论可靠。

五、模型评价与推广

5.1 模型优点

（1）命中判定几何精确：以长方体最近距离刻画杀伤判定，正确处理了平面内触发引信、平面内定深引信与平面外定深引信三种情形，避免了将杀伤范围简单近似为球心距离带来的偏差。

（2）解析与数值双重验证：问题一、问题二均给出闭式或积分表达式，并与大规模蒙特卡洛模拟交叉验证（相对偏差均小于 0.2%），问题三亦以大样本最终验证，结果可靠性高。

（3）多弹相关性建模正确：问题三通过对潜艇位置积分求条件并集，正确刻画了多弹命中事件的相关结构，并定量揭示独立近似的低估偏差（约 4.0 个百分点），具有方法上的创新性。

（4）模型可扩展性强：阵列构型、弹数、误差参数均可作为输入调整，灵敏度分析完备，可为反潜火力规划提供量化依据。

（5）方法通用性好：模型仅依赖命中判定几何与误差分布函数，改变阵列构型、弹数或误差类型仅需替换相应模块，可快速适配不同反潜任务场景。

（6）计算与验证分离：解析积分负责精确求值与寻优，蒙特卡洛负责独立验证，二者使用相互独立的随机源，避免了系统性偏差的隐蔽传递。

5.2 模型缺点

（1）未考虑深弹投放自身的落点散布、弹体入水后运动轨迹偏差，实际命中概率可能略低于模型值。

（2）假设潜艇在攻击瞬间静止且完全静默，未考虑潜艇机动规避与诱饵干扰。

（3）杀伤区以理想球体近似，未计及水中爆炸对杀伤半径的方向性衰减。

（4）问题三采用蒙特卡洛+网格搜索求解，计算量较大，且搜索结果受样本共享引入的统计偏倚影响，未给出解析梯度与严格收敛性证明。

（5）灵敏度分析以单参数扰动为主，未系统考察多参数联合扰动（如水平与深度误差同时恶化）的耦合效应，后续可引入正交试验或全局灵敏度分析方法加以完善。

5.3 模型推广

本文方法可直接推广至任意阵列构型（矩形阵、圆形阵、随机散布）、任意弹数与分层定深方案，亦可用于鱼雷齐射、多机协同搜索等更一般的反潜武器运用优化。以弹数扩展为例，在最优定深 $d^*=158.75\text{m}$、朝向 $\theta=45°$、间隔 $\Delta^*=85\text{m}$ 下，将 3×3 方阵扩为 4×4 方阵（16 枚深弹）后，至少一枚命中概率由 0.3629 升至约 0.5028（3×10^6样本），相对提升约 38.5%，表明弹药允许时方阵扩展可显著增强打击效果，且在沿用 3×3 最优参数的条件下命中概率仍有显著提升，说明方阵扩展对参数偏差具有鲁棒性，方法具有良好的可移植性。将潜艇机动模型、深弹弹道散布纳入后，可进一步发展为动态火力规划与打击效能评估系统。

5.4 模型检验与稳健性

（1）解析与模拟一致性：问题一、问题二的解析结果与蒙特卡洛模拟的相对偏差分别约 0.1%与 0.16%（见表 2、表 3），问题三最优方案经 1.2×10^{7}样本最终验证，概率估计的标准误差约 1.4×10^{-4}，三问结果均达到统计一致性要求。

（2）样本量收敛性：蒙特卡洛估计的误差随样本量按 $N^{(-1/2)}$衰减，符合大数定律与中心极限定理的预期[4]；问题一在 8×10^{6}样本下模拟值与解析值的偏差约 9×10^{-5}，处于两倍标准误差以内，证明模拟设置正确且样本量充分[8]。

（3）搜索与离散化稳健性：问题三采用粗扫描—精扫描—最终验证三层递进搜索，各层最优参数与命中概率逐级一致；问题一的区域分解积分、问题二的极坐标变换积分均通过独立分解或独立方法交叉验证，不存在明显的离散化偏差。

（4）适用范围与局限性：本文模型适用于潜艇静止、深弹垂直下落、杀伤区为球体的理想化场景。模型的输入为潜艇几何参数、定位误差分布与深弹杀伤半径，输出为命中概率与最优投弹方案，可直接应用于海峡、浅海等海底地形复杂海域的反潜作战规划。当潜艇处于机动状态或深弹存在投放散布时，需将潜艇位移与投放误差纳入模型，此时命中概率将有所降低，但模型框架仍可复用。灵敏度分析表明，命中概率对水平定位精度最敏感（σ 减半可使 P^{*}提高一倍以上），因此在实际作战中应优先提高定位精度，其次优化弹药参数与投放方案。

六、结论

本文针对反潜航空深弹命中概率问题，建立了由浅入深、环环相扣的概率模型体系，主要结论如下：

（1）深度无误差情形：最优投弹落点为定位投影原点，定深引信深度取潜艇上、下表面之间的任意值（[137.5,162.5]m）即可获得最大命中概率 $P^{*}\approx0.0837$；定深存在宽达 25m 的最优平台，落点偏移 20m 内概率仅下降约 1.2%，对小幅偏差不敏感，方案稳健。

（2）三维均有误差情形：命中概率关于定深呈单峰结构，存在唯一最优定深 $d^{*}\approx157.84$m，对应最大命中概率 $P^{*}\approx0.0576$；深度定位误差使单弹命中率相对问题一下降约 31%，是影响单弹命中率的主要因素，提高深度定位精度可显著改善命中效果。

（3）多弹阵列协同情形：3×3 方阵的最优方案为 $d^{*}=158.75$m、$\Delta^{*}=85$m、$\theta^{*}=45°$，至少一枚命中概率 $P^{*}\approx0.3632$，较单弹相对提升约 5.3 倍（提高到约 6.3

倍）；联合建模的概率显著高于独立近似（0.3237），多弹命中事件的相关性不可忽略，独立近似公式不能作为多弹方案的评估依据。

模型全部结论均经解析推导与大规模蒙特卡洛模拟交叉验证，灵敏度分析表明最优方案对定深、落点、误差与阵列参数均具有良好稳健性，可为反潜火力规划提供量化依据。

参考文献

[1]姜启源，谢金星，叶俊.数学模型（第五版）[M].北京：高等教育出版社，2018.

[2]盛骤，谢式千，潘承毅.概率论与数理统计（第四版）[M].北京：高等教育出版社，2008.

[3]李庆扬，王能超，易大义.数值分析（第五版）[M].北京：清华大学出版社，2008.

[4]KroeseDP,BreretonT,TaimreT,etal.WhytheMonteCarlomethodissoimportanttoday[J].WIREsComputationalStatistics,2014,6(6):386-392.

[5]VirtanenP,GommersR,OliphantTE,etal.SciPy1.0:fundamentalalgorithmsforscientificcomputinginPython[J].NatureMethods,2020,17(3):261-272.

[6]张最良，李长生，赵文志，等.军事运筹学[M].北京：军事科学出版社，2002.

[7]HunterJD.Matplotlib:A2Dgraphicsenvironment[J].ComputinginScience&Engineering,2007,9(3):90-95.

[8]康崇禄.蒙特卡罗方法及应用[M].北京：国防工业出版社，2007.

附录

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、公式排版辅助、代码调试与验证，详细使用情况见支撑材料。

以下为三个问题与灵敏度分析的完整可运行求解代码。运行环境：Python3.10+，依赖 numpy、scipy、matplotlib；在项目根目录下依次执行 pythoncode/solve_q1.py、pythoncode/solve_q2.py、pythoncode/solve_q3.py、pythoncode/charts_data.py、pythoncode/charts.py、pythoncode/sensitivity.py，即可复现全部数值结果、图表数据与灵敏度表格（论文表 5~表 9 由 sensitivity.py 生成）；图表脚本 charts_data.py、charts.py 因篇幅未在附录中展开，见支撑材料 code/目录。

问题一求解代码（solve_q1.py）

```
#-*-coding:utf-8-*-
```

"""问题一：投射一枚深弹，深度定位无误差。
推导最大命中概率与(落点,引爆深度)关系，给出最优方案与最大命中概率。
"""
importjson,sys,io
importnumpyasnp
fromscipyimportintegrate,stats
sys.stdout=io.TextIOWrapper(sys.stdout.buffer,encoding="utf-8")

#----------------参数----------------
withopen(r"data/parameters.json",encoding="utf-8-sig")asf:
P=json.load(f)
a=P["submarine"]["half_length"]#50m
b=P["submarine"]["half_width"]#10m
c=P["submarine"]["half_height"]#12.5m
R=P["weapon"]["kill_radius"]#20m
sigma=P["errors"]["sigma_xy"]#120m
h0=P["errors"]["depth_est"]#150m
z_up=h0-c#潜艇上表面 137.5
z_lo=h0+c#潜艇下表面 162.5

phi=stats.norm.pdf
Phi=stats.norm.cdf

#----------------命中逻辑（单次投弹，Z=h0 固定）----------------
defhit_mask(X,Y,x0,y0,d):
"""给定潜艇中心位置(X,Y)（深度固定 h0），判断深弹(x0,y0,d)是否命中。"""
dx=np.abs(X-x0);dy=np.abs(Y-y0)
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0)**2+np.maximum(dy-b,0)**2)
v=np.maximum(z_up-d,np.maximum(d-z_lo,0.0))
dist=np.sqrt(h**2+v**2)
hit=np.zeros_like(X,dtype=bool)
hit|=in_plane&(d>=z_up)#情形 1：触发引信
hit|=in_plane&(d<z_up)&((z_up-d)<=R)#情形 2：定深、平面内
hit|=(~in_plane)&(dist<=R)#情形 3：定深、平面外
returnhit

#----------------闭式表达式----------------
#平面内命中概率（原点投弹）
p_in=(2*Phi(a/sigma)-1)*(2*Phi(b/sigma)-1)

#P(B)=P(max(|X|-a,0)^2+max(|Y|-b,0)^2<=R^2)，即触发区∪定深平面外区（d∈[z_up,z_lo]时）
#第一象限数值积分：x∈[0,a+R],y∈[0,b+R]
defintegrand(y,x):
if(max(x-a,0.0))**2+(max(y-b,0.0))**2<=R**2:
returnphi(x/sigma)/sigma*phi(y/sigma)/sigma
return0.0

pB,pB_err=integrate.dblquad(integrand,0.0,a+R,
lambdax:0.0,lambdax:b+R,epsabs=1e-12,epsrel=1e-12)
pB*=4.0

#角区（平面外、带状之外、四分之一圆）单独分解（供论文表格）
#区域分解：
#B1=[-a,a]x[-b-R,b+R]平面内 X+Y 外扩带
#B2=[-a-R,a+R]x[-b,b]X 外扩带+平面内 Y
#B0=B1∩B2=平面内

```
#B3=角区
pB1=(2*Phi(a/sigma)-1)*(2*Phi((b+R)/sigma)-1)
pB2=(2*Phi((a+R)/sigma)-1)*(2*Phi(b/sigma)-1)
pB0=p_in
#角区：第一象限 x∈(a,a+R],y∈(b,b+R],圆约束
defintegrand_corner(y,x):
if(x-a)**2+(y-b)**2<=R**2:
returnphi(x/sigma)/sigma*phi(y/sigma)/sigma
return0.0
pB3,_=integrate.dblquad(integrand_corner,a,a+R,
lambdax:b,lambdax:b+np.sqrt(R**2-(x-a)**2),
epsabs=1e-12,epsrel=1e-12)
pB3*=4.0
pB_decomp=pB1+pB2-pB0+pB3

#----------------蒙特卡洛验证（原点投弹）----------------
rng=np.random.default_rng(2026)
N=8_000_000
X=rng.normal(0.0,sigma,N)
Y=rng.normal(0.0,sigma,N)
h=np.sqrt(np.maximum(np.abs(X)-a,0)**2+np.maximum(np.abs(Y)-b,0)**2)

#直接蒙特卡洛：d 取在[z_up,z_lo]内（v=0）
mc_d150=hit_mask(X,Y,0.0,0.0,h0).mean()
#P(B)蒙特卡洛
mc_pB=(h<=R).mean()
#平面内
mc_pin=((np.abs(X)<=a)&(np.abs(Y)<=b)).mean()

#----------------d 敏感性（扫描 d，蒙特卡洛估计命中概率）----------------
d_grid=np.linspace(90.0,200.0,23)
p_d=np.array([hit_mask(X,Y,0.0,0.0,d).mean()fordind_grid])
d_opt=d_grid[np.argmax(p_d)]

#----------------落点 x0 敏感性（d=150 固定）----------------
x0_grid=np.linspace(0.0,100.0,11)
p_x0=np.array([hit_mask(X,Y,x0,0.0,h0).mean()forx0inx0_grid])

#----------------结果输出----------------
res={
"P_in_plane":p_in,#仅触发引信（平面内）
"P_max_analytic":pB,#最优命中概率（数值积分）
"P_max_analytic_decomp":pB_decomp,
"P_max_analytic_err":pB_err,
"MC_hit_d150":mc_d150,
"MC_PB":mc_pB,
"MC_pin":mc_pin,
"P_B1":pB1,"P_B2":pB2,"P_B0":pB0,"P_B3":pB3,
"d_opt_interval":[z_up,z_lo],
"d_sensitivity":{"d":d_grid.tolist(),"p":p_d.tolist()},
"x0_sensitivity":{"x0":x0_grid.tolist(),"p":p_x0.tolist()},
"opt_scheme":{"x0":0.0,"y0":0.0,"d":h0},
}
withopen(r"output/q1/result_q1.json","w",encoding="utf-8")asf:
json.dump(res,f,ensure_ascii=False,indent=2)

print("===问题一结果===")
```

```
print(f"平面内概率 P(D1)={p_in:.6f}")
print(f"最大命中概率 P*(数值积分)={pB:.6f}(err={pB_err:.2e})")
print(f"P*分解验证(B1+B2-B0+B3)={pB_decomp:.6f}")
print(f"其中 B1=平面内 X×Y 外扩带={pB1:.6f}")
print(f"B2=X 外扩带×平面内 Y={pB2:.6f}")
print(f"B0=平面内={pB0:.6f}")
print(f"B3=角区(四分之一圆)={pB3:.6f}")
print(f"蒙特卡洛:d=150 命中概率={mc_d150:.6f}")
print(f"蒙特卡洛:P(h<=R)={mc_pB:.6f}")
print(f"蒙特卡洛:平面内={mc_pin:.6f}")
print(f"最优引爆深度区间 d*=[{z_up},{z_lo}]m")
print(f"扫描确认最优 d(格点)={d_opt:.1f}m,P={p_d.max():.6f}")
print(f"落点扫描:x0=0 最优 P={p_x0[0]:.6f};x0=100 时 P={p_x0[-1]:.6f}")
```

问题二求解代码（solve_q2.py）

```
#-*-coding:utf-8-*-
"""问题二：三坐标均有误差（Z 单边截尾正态），给出命中概率表达式并优化定深深度 d。"""
importjson,sys,io
importnumpyasnp
fromscipyimportintegrate,stats,optimize
sys.stdout=io.TextIOWrapper(sys.stdout.buffer,encoding="utf-8")

withopen(r"data/parameters.json",encoding="utf-8-sig")asf:
P=json.load(f)
a,b,c=P["submarine"]["half_length"],P["submarine"]["half_width"],P["submarine"]["half_height"]
R=P["weapon"]["kill_radius"]
sigma=P["errors"]["sigma_xy"]
h0=P["errors"]["depth_est"]
sigz=P["errors"]["sigma_z"]
l=P["errors"]["min_depth"]

phi=stats.norm.pdf
Phi=stats.norm.cdf

#----------------截尾正态 Z 的 CDF----------------
denom=1.0-Phi((l-h0)/sigz)#=Phi((h0-l)/sigz)≈0.7734
defF_Z(z):
z=np.asarray(z,dtype=float)
returnnp.where(z<=l,0.0,(Phi((z-h0)/sigz)-Phi((l-h0)/sigz))/denom)

deff_X(x):returnphi(x/sigma)/sigma
deff_Y(y):returnphi(y/sigma)/sigma

#----------------命中逻辑（蒙特卡洛）----------------
defhit_mask_q2(X,Y,Z,x0,y0,d):
dx=np.abs(X-x0);dy=np.abs(Y-y0)
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0)**2+np.maximum(dy-b,0)**2)
v=np.maximum(Z-c-d,np.maximum(d-Z-c,0.0))
dist=np.sqrt(h**2+v**2)
hit=np.zeros_like(X,dtype=bool)
hit|=in_plane&(Z<=d+c)#E1 触发引信
hit|=in_plane&(Z>d+c)&((Z-c-d)<=R)#E2 定深、平面内
hit|=(~in_plane)&(dist<=R)#E3 定深、平面外
returnhit
```

```
defsample_Z(rng,N):
u=rng.uniform(0.0,1.0,N)
returnh0+sigz*stats.norm.ppf(Phi((1-h0)/sigz)+u*denom)

#----------------精确积分 P(d)----------------
defP_integral(d):
#第一项：平面内，Z<=d+c+R
term1=(2*Phi(a/sigma)-1)*(2*Phi(b/sigma)-1)*F_Z(d+c+R)
#第二项：平面外 h<=R，第一象限三分区 x4
#B1:x∈[0,a],y∈(b,b+R],s=y-b
definteg_B1(y,x):
s=np.sqrt(R**2-(y-b)**2)
return(F_Z(d+c+s)-F_Z(d-c-s))*f_X(x)*f_Y(y)
IB1,_=integrate.dblquad(integ_B1,0.0,a,b,b+R,
epsabs=1e-10,epsrel=1e-9)
#B2:x∈(a,a+R],y∈[0,b],s=x-a
definteg_B2(y,x):
s=np.sqrt(R**2-(x-a)**2)
return(F_Z(d+c+s)-F_Z(d-c-s))*f_X(x)*f_Y(y)
IB2,_=integrate.dblquad(integ_B2,a,a+R,0.0,b,
epsabs=1e-10,epsrel=1e-9)
#B3:角区，极坐标 x=a+rcosθ,y=b+rsinθ，r∈(0,R]，θ∈(0,π/2]
definteg_B3_polar(theta,r):
x=a+r*np.cos(theta)
y=b+r*np.sin(theta)
s=np.sqrt(R**2-r**2)
return(F_Z(d+c+s)-F_Z(d-c-s))*f_X(x)*f_Y(y)*r
IB3,_=integrate.dblquad(integ_B3_polar,0.0,R,0.0,np.pi/2,
epsabs=1e-10,epsrel=1e-9)
returnterm1+4.0*(IB1+IB2+IB3),(term1,4*IB1,4*IB2,4*IB3)

#----------------蒙特卡洛粗扫描 d----------------
rng=np.random.default_rng(2026)
Nmc=2_000_000
Xmc=rng.normal(0.0,sigma,Nmc)
Ymc=rng.normal(0.0,sigma,Nmc)
Zmc=sample_Z(rng,Nmc)

d_grid=np.linspace(100.0,190.0,37)
P_mc_grid=np.array([hit_mask_q2(Xmc,Ymc,Zmc,0.0,0.0,d).mean()fordind_grid])
d_mc_best=d_grid[np.argmax(P_mc_grid)]

#----------------精确优化（Brent，在粗最优附近）----------------
lo=max(100.0,d_mc_best-15.0)
hi=min(200.0,d_mc_best+15.0)
defnegP(d):return-P_integral(d)[0]
opt=optimize.minimize_scalar(negP,bounds=(lo,hi),method="bounded",
options={"xatol":1e-6})
d_star=opt.x
P_star,comp=P_integral(d_star)

#----------------验证----------------
Nv=8_000_000
Xv=rng.normal(0.0,sigma,Nv)
Yv=rng.normal(0.0,sigma,Nv)
Zv=sample_Z(rng,Nv)
```

```
P_mc_star=hit_mask_q2(Xv,Yv,Zv,0.0,0.0,d_star).mean()

#落点偏移验证（x0 扫描，d=d_star）
P_mc_x0=[hit_mask_q2(Xv,Yv,Zv,x0,0.0,d_star).mean()forx0in(0,30,60,100)]

#----------------灵敏度：σz 与 l----------------
defF_Z_g(z,sz):
z=np.asarray(z,dtype=float)
den=1.0-Phi((l-h0)/sz)
returnnp.where(z<=l,0.0,(Phi((z-h0)/sz)-Phi((l-h0)/sz))/den)

defsample_Z_g(rng,N,sz):
u=rng.uniform(0.0,1.0,N)
returnh0+sz*stats.norm.ppf(Phi((l-h0)/sz)+u*(1-Phi((l-h0)/sz)))

sens=[]
forsigz_vin(30.0,40.0,50.0):
dgrid=np.linspace(110.0,180.0,15)
Pmc=[]
fordindgrid:
Zs=sample_Z_g(rng,500_000,sigz_v)
Xs=rng.normal(0.0,sigma,500_000);Ys=rng.normal(0.0,sigma,500_000)
Pmc.append(hit_mask_q2(Xs,Ys,Zs,0.0,0.0,d).mean())
sens.append({"sigz":sigz_v,"d_best":float(dgrid[int(np.argmax(Pmc))]),
"P_best":float(np.max(Pmc))})

#----------------输出----------------
res={
"P_expression_terms":{
"E1_E2_plane_in":"P(D1)*F_Z(d+c+R)",
"E3_plane_out":"int_{D2,h<=R}[F_Z(d+c+s)-F_Z(d-c-s)]f_Xf_Ydxdy",
"c":c,"R":R
},
"d_grid_mc":d_grid.tolist(),"P_mc_grid":P_mc_grid.tolist(),
"d_mc_best":float(d_mc_best),
"d_star":float(d_star),"P_star_integral":float(P_star),
"P_star_components":{"plane_in":comp[0],"B1":comp[1],"B2":comp[2],"B3":comp[3]},
"P_star_mc":float(P_mc_star),
"P_mc_x0":{"x0":[0,30,60,100],"p":P_mc_x0},
"sensitivity_sigz":sens,
}
withopen(r"output/q2/result_q2.json","w",encoding="utf-8")asf:
json.dump(res,f,ensure_ascii=False,indent=2)

print("===问题二结果===")
print(f"蒙特卡洛粗扫描最优 d={d_mc_best:.1f},P={P_mc_grid.max():.6f}")
print(f"精确最优 d*={d_star:.2f}m")
print(f"P(d*)精确积分={P_star:.6f}")
print(f"平面内(触发+定深)={comp[0]:.6f}")
print(f"平面外 B1(X 内 Y 带)={comp[1]:.6f}")
print(f"平面外 B2(X 带 Y 内)={comp[2]:.6f}")
print(f"平面外 B3(角区)={comp[3]:.6f}")
print(f"P(d*)蒙特卡洛(8M)={P_mc_star:.6f}")
print(f"落点偏移验证(0,30,60,100)={[f'{p:.6f}'forpinP_mc_x0]}")
print("σz 灵敏度:",[(s['sigz'],round(s['d_best'],1),round(s['P_best'],5))forsinsens])
```

问题三求解代码（solve_q3.py）

```
#-*-coding:utf-8-*-
"""问题三：9 枚深弹 3×3 方阵，同深度 d，间隔 Δ，朝向 θ，最大化至少一枚命中概率。
对每个朝向角 θ ∈{0,15,30,45}在(d,Δ)平面上做完整二维分层网格搜索（粗扫描+精扫描），
比较各朝向局部最优取全局最优，再以大样本验证；多弹命中事件对同一潜艇位置条件相关，
须对潜艇位置积分求并集概率。
"""
importjson,sys,io,time
importnumpyasnp
fromscipyimportstats
sys.stdout=io.TextIOWrapper(sys.stdout.buffer,encoding="utf-8")


withopen(r"data/parameters.json",encoding="utf-8-sig")asf:
P=json.load(f)
a,b,c=P["submarine"]["half_length"],P["submarine"]["half_width"],P["submarine"]["half_height"]
R=P["weapon"]["kill_radius"]
sigma=P["errors"]["sigma_xy"];h0=P["errors"]["depth_est"]
sigz=P["errors"]["sigma_z"];l=P["errors"]["min_depth"]


phi=stats.norm.pdf;Phi=stats.norm.cdf
denom=1.0-Phi((l-h0)/sigz)


defsample_XYZ(rng,N):
X=rng.normal(0.0,sigma,N)
Y=rng.normal(0.0,sigma,N)
u=rng.uniform(0.0,1.0,N)
Z=h0+sigz*stats.norm.ppf(Phi((l-h0)/sigz)+u*denom)
returnX,Y,Z


defarray_points(delta,theta_deg):
"""3×3 方阵落点（中心在原点），绕 Z 轴旋转 theta，返回 x/y 向量。"""
th=np.deg2rad(theta_deg)
off=np.array([-delta,0.0,delta])
xs,ys=np.meshgrid(off,off)
xr=xs*np.cos(th)-ys*np.sin(th)
yr=xs*np.sin(th)+ys*np.cos(th)
returnxr.ravel(),yr.ravel()


defprob_any_hit(X,Y,Z,px,py,d,chunk=1_000_000):
"""向量化计算“至少一枚命中”频率（分块控制内存）。"""
n=len(X);total=0
forsinrange(0,n,chunk):
e=min(s+chunk,n)
Xs=X[s:e];Ys=Y[s:e];Zs=Z[s:e]
dx=np.abs(Xs[:,None]-px[None,:]);dy=np.abs(Ys[:,None]-py[None,:])
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0.0)**2+np.maximum(dy-b,0.0)**2)
v=np.maximum(Zs[:,None]-c-d,np.maximum(d-Zs[:,None]-c,0.0))
dist=np.sqrt(h**2+v**2)
hit=in_plane&(Zs[:,None]<=d+c)
hit|=in_plane&(Zs[:,None]>d+c)&((Zs[:,None]-c-d)<=R)
```

```
hit|=(~in_plane)&(dist<=R)
total+=int(hit.any(axis=1).sum())
returntotal/n

defprob_ind_approx(X,Y,Z,px,py,d,chunk=1_000_000):
"""独立近似：把各弹命中当作独立事件（会低估）。"""
n=len(X);cnt=np.zeros(len(px),dtype=np.float64)
forsinrange(0,n,chunk):
e=min(s+chunk,n)
Xs=X[s:e];Ys=Y[s:e];Zs=Z[s:e]
dx=np.abs(Xs[:,None]-px[None,:]);dy=np.abs(Ys[:,None]-py[None,:])
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0.0)**2+np.maximum(dy-b,0.0)**2)
v=np.maximum(Zs[:,None]-c-d,np.maximum(d-Zs[:,None]-c,0.0))
dist=np.sqrt(h**2+v**2)
hit=in_plane&(Zs[:,None]<=d+c)
hit|=in_plane&(Zs[:,None]>d+c)&((Zs[:,None]-c-d)<=R)
hit|=(~in_plane)&(dist<=R)
cnt+=hit.sum(axis=0)
p_i=cnt/n
return1.0-np.prod(1.0-p_i)

rng=np.random.default_rng(42)

#----------分层网格搜索：每个朝向角θ做完整二维重扫----------
d_grid=np.linspace(140.0,175.0,25)
del_grid=np.linspace(40.0,170.0,27)
thetas=[0.0,15.0,30.0,45.0]
Nc=500_000
Nf=2_000_000

t0=time.time()
Xc,Yc,Zc=sample_XYZ(rng,Nc)
coarse_res={}
forthinthetas:
Pmat=np.zeros((len(d_grid),len(del_grid)))
fori,dinenumerate(d_grid):
forj,dlinenumerate(del_grid):
px,py=array_points(dl,th)
Pmat[i,j]=prob_any_hit(Xc,Yc,Zc,px,py,d)
coarse_res[th]=Pmat
i0,j0=np.unravel_index(np.argmax(Pmat),Pmat.shape)
print(f"粗扫描θ={th}° :P={Pmat.max():.4f}atd={d_grid[i0]:.1f}, Δ={del_grid[j0]:.0f}",flush=True)

Xf,Yf,Zf=sample_XYZ(rng,Nf)
fine_res={}
theta_scan=[]
forthinthetas:
Pmat=coarse_res[th]
i0,j0=np.unravel_index(np.argmax(Pmat),Pmat.shape)
d_lo,d_hi=d_grid[max(i0-2,0)],d_grid[min(i0+2,len(d_grid)-1)]
dl_lo,dl_hi=del_grid[max(j0-2,0)],del_grid[min(j0+2,len(del_grid)-1)]
d_fine=np.linspace(d_lo,d_hi,15)
dl_fine=np.linspace(dl_lo,dl_hi,15)
Pfine=np.zeros((len(d_fine),len(dl_fine)))
fori,dinenumerate(d_fine):
forj,dlinenumerate(dl_fine):
px,py=array_points(dl,th)
```

```
Pfine[i,j]=prob_any_hit(Xf,Yf,Zf,px,py,d)
i1,j1=np.unravel_index(np.argmax(Pfine),Pfine.shape)
fine_res[th]={"d":d_fine.tolist(),"delta":dl_fine.tolist(),"P":Pfine.tolist()}
theta_scan.append({"theta":th,"P":float(Pfine.max()),
"d":float(d_fine[i1]),"delta":float(dl_fine[j1])})
print(f"精扫描θ={th}°:P={Pfine.max():.5f}atd={d_fine[i1]:.2f},Δ={dl_fine[j1]:.2f}",flush=True)


best=max(theta_scan,key=lambdat:t["P"])
th_opt,d_opt,dl_opt=best["theta"],best["d"],best["delta"]
print(f"全局最优:θ={th_opt:.0f}°,d={d_opt:.2f},Δ={dl_opt:.2f}",flush=True)

#----------大样本最终验证----------
Nv=12_000_000
Xv,Yv,Zv=sample_XYZ(rng,Nv)
px_opt,py_opt=array_points(dl_opt,th_opt)
P_joint=prob_any_hit(Xv,Yv,Zv,px_opt,py_opt,d_opt)
P_ind=prob_ind_approx(Xv,Yv,Zv,px_opt,py_opt,d_opt)
#单弹命中率（原点投弹）
px1,py1=array_points(0.0,0.0)
p1=prob_any_hit(Xv,Yv,Zv,px1,py1,d_opt)
print(f"\n最终方案:d*={d_opt:.2f}m,Δ*={dl_opt:.2f}m,θ*={th_opt:.0f}°")
print(f"单弹命中率p1={p1:.5f}")
print(f"至少一枚命中P_joint={P_joint:.5f}（大样本12M）")
print(f"独立近似P_ind={P_ind:.5f}（低估{P_joint-P_ind:.5f}）")

#----------保存----------
res={
"coarse":{"d":d_grid.tolist(),"delta":del_grid.tolist(),
"P":coarse_res[0.0].tolist()},
"fine":{str(int(t)):vfort,vinfine_res.items()},
"theta_scan":[{"theta":t["theta"],"P":t["P"],"d":t["d"],"delta":t["delta"]}
fortintheta_scan],
"opt":{"d":d_opt,"delta":dl_opt,"theta_deg":th_opt,
"P_joint":P_joint,"P_independent_approx":P_ind,"p_single":p1},
}
withopen(r"output/q3/result_q3.json","w",encoding="utf-8")asf:
json.dump(res,f,ensure_ascii=False,indent=2)
print(f"savedresult_q3.json（总耗时{time.time()-t0:.0f}s）")
```

灵敏度分析代码（sensitivity.py）

```
#-*-coding:utf-8-*-
"""灵敏度分析与补充统计：统一生成论文表5（Δ灵敏度）、表7（σ/R灵敏度）、
表8（σ_z灵敏度）、表9（收敛性）、4×4方阵扩展与多弹相关性统计。
运行方式：pythoncode/sensitivity.py（在项目根目录下）。
积分采用48点高斯-勒让德求积，与scipy自适应积分一致（偏差<1e-6）。
"""
importjson,sys,io,time
importnumpyasnp
fromscipyimportstats,optimize
fromnumpy.polynomial.legendreimportleggauss
sys.stdout=io.TextIOWrapper(sys.stdout.buffer,encoding="utf-8")


withopen(r"data/parameters.json",encoding="utf-8-sig")asf:
P=json.load(f)
a,b,c=P["submarine"]["half_length"],P["submarine"]["half_width"],P["submarine"]["half_height"]
R0=P["weapon"]["kill_radius"]
sigma0=P["errors"]["sigma_xy"]
```

```
h0=P["errors"]["depth_est"]
sigz0=P["errors"]["sigma_z"]
l=P["errors"]["min_depth"]

phi=stats.norm.pdf;Phi=stats.norm.cdf
xg,wg=leggauss(48)

defquad2_rect(f,x_lo,x_hi,y_lo,y_hi):
x=0.5*(x_hi-x_lo)*xg+0.5*(x_hi+x_lo)
y=0.5*(y_hi-y_lo)*xg+0.5*(y_hi+y_lo)
XX,YY=np.meshgrid(x,y)
fv=f(XX,YY)
return0.25*(x_hi-x_lo)*(y_hi-y_lo)*np.sum(fv*wg[None,:]*wg[:,None])

#============表 7：问题一 σ/R 灵敏度（精确积分）============
defPmax_q1(sigma_v,R_v):
#按区域分解避免积分边界不连续：P(B)=B1+B2-B0+B3（角区用极坐标光滑积分）
A1=2*Phi(a/sigma_v)-1;B1=2*Phi(b/sigma_v)-1
AB=2*Phi((a+R_v)/sigma_v)-1;BB=2*Phi((b+R_v)/sigma_v)-1
pB1=A1*BB;pB2=AB*B1;pB0=A1*B1
r=0.5*R_v*xg+0.5*R_v
th=0.5*(np.pi/2)*xg+0.5*(np.pi/2)
RR,TH=np.meshgrid(r,th)
X=a+RR*np.cos(TH);Y=b+RR*np.sin(TH)
f=phi(X/sigma_v)/sigma_v*phi(Y/sigma_v)/sigma_v*RR
pB3_1q=0.25*R_v*(np.pi/2)*np.sum(f*wg[None,:]*wg[:,None])
returnpB1+pB2-pB0+4.0*pB3_1q

tbl7=[]
forsin(60,80,100,120,140,160):
tbl7.append({"param":"sigma","value":s,"P":Pmax_q1(s,R0)})
forrin(15,20,25,30):
tbl7.append({"param":"R","value":r,"P":Pmax_q1(sigma0,r)})

#============表 8：问题二 σ_z 灵敏度（精确积分+一维寻优）============
defF_Z_g(z,sz):
z=np.asarray(z,dtype=float)
den=1.0-Phi((l-h0)/sz)
returnnp.where(z<=l,0.0,(Phi((z-h0)/sz)-Phi((l-h0)/sz))/den)

defP_integral_q2(d,sz):
deffx(x):returnphi(x/sigma0)/sigma0
deffy(y):returnphi(y/sigma0)/sigma0
term1=(2*Phi(a/sigma0)-1)*(2*Phi(b/sigma0)-1)*F_Z_g(d+c+R0,sz)
deffB1(X,Y):
s=np.sqrt(R0**2-(Y-b)**2)
return(F_Z_g(d+c+s,sz)-F_Z_g(d-c-s,sz))*fx(X)*fy(Y)
IB1=quad2_rect(fB1,0.0,a,b,b+R0)
deffB2(X,Y):
s=np.sqrt(R0**2-(X-a)**2)
return(F_Z_g(d+c+s,sz)-F_Z_g(d-c-s,sz))*fx(X)*fy(Y)
IB2=quad2_rect(fB2,a,a+R0,0.0,b)
r=0.5*R0*xg+0.5*R0
th=0.5*(np.pi/2)*xg+0.5*(np.pi/2)
RR,TH=np.meshgrid(r,th)
X=a+RR*np.cos(TH);Y=b+RR*np.sin(TH)
s=np.sqrt(R0**2-RR**2)
```

```
fB3=(F_Z_g(d+c+s,sz)-F_Z_g(d-c-s,sz))*fx(X)*fy(Y)*RR
IB3=0.25*R0*(np.pi/2)*np.sum(fB3*wg[None,:]*wg[:,None])
returnterm1+4.0*(IB1+IB2+IB3)

tbl8=[]
forszin(20,30,40,50,60):
opt=optimize.minimize_scalar(lambdad:-P_integral_q2(d,sz),bounds=(100.0,185.0),
method="bounded",options={"xatol":1e-4})
tbl8.append({"sigz":sz,"d_star":float(opt.x),"P_star":float(-opt.fun)})

#============表 9：问题一收敛性（d=150m 原点投弹）============
defhit_q1(X,Y,d):
dx=np.abs(X);dy=np.abs(Y)
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0)**2+np.maximum(dy-b,0)**2)
v=np.maximum((h0-c)-d,np.maximum(d-(h0+c),0.0))
dist=np.sqrt(h**2+v**2)
hit=in_plane&(d>=(h0-c))
hit|=in_plane&(d<(h0-c))&(((h0-c)-d)<=R0)
hit|=(~in_plane)&(dist<=R0)
returnhit

P_analytic=0.0837350141373228
tbl9=[]
rng9=np.random.default_rng(2026)
forNin(10_000,100_000,1_000_000,8_000_000):
X=rng9.normal(0.0,sigma0,N);Y=rng9.normal(0.0,sigma0,N)
freq=hit_q1(X,Y,h0).mean()
se=np.sqrt(freq*(1-freq)/N)
tbl9.append({"N":N,"freq":float(freq),"dev":float(freq-P_analytic),"se":float(se)})

#============表 5：问题三Δ灵敏度（θ=45°，d=d*）============
q3=json.load(open(r"output/q3/result_q3.json",encoding="utf-8"))
d_opt=q3["opt"]["d"];dl_opt=q3["opt"]["delta"];th_opt=q3["opt"]["theta_deg"]
denom=1.0-Phi((l-h0)/sigz0)

defsample_XYZ3(rng,N):
X=rng.normal(0.0,sigma0,N);Y=rng.normal(0.0,sigma0,N)
u=rng.uniform(0.0,1.0,N)
Z=h0+sigz0*stats.norm.ppf(Phi((l-h0)/sigz0)+u*denom)
returnX,Y,Z

defarray_points(delta,theta_deg):
th=np.deg2rad(theta_deg);off=np.array([-delta,0.0,delta])
xs,ys=np.meshgrid(off,off)
return(xs*np.cos(th)-ys*np.sin(th)).ravel(),(xs*np.sin(th)+ys*np.cos(th)).ravel()

defprob_any_hit(X,Y,Z,px,py,d,chunk=1_000_000):
n=len(X);total=0
forsinrange(0,n,chunk):
e=min(s+chunk,n)
Xs=X[s:e];Ys=Y[s:e];Zs=Z[s:e]
dx=np.abs(Xs[:,None]-px[None,:]);dy=np.abs(Ys[:,None]-py[None,:])
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0.0)**2+np.maximum(dy-b,0.0)**2)
v=np.maximum(Zs[:,None]-c-d,np.maximum(d-Zs[:,None]-c,0.0))
dist=np.sqrt(h**2+v**2)
```

```
hit=in_plane&(Zs[:,None]<=d+c)
hit|=in_plane&(Zs[:,None]>d+c)&((Zs[:,None]-c-d)<=R0)
hit|=(~in_plane)&(dist<=R0)
total+=int(hit.any(axis=1).sum())
returntotal/n

rng3=np.random.default_rng(7)
N3=2_000_000
X3,Y3,Z3=sample_XYZ3(rng3,N3)
deltas=[50.0,70.0,dl_opt,110.0,130.0,150.0]
tbl5=[]
fordlindeltas:
px,py=array_points(dl,th_opt)
pv=prob_any_hit(X3,Y3,Z3,px,py,d_opt)
tbl5.append({"delta":dl,"P":float(pv)})
P_opt_dl=tbl5[[round(t["delta"],2)fortintbl5].index(round(dl_opt,2))]["P"]

#============4×4 方阵扩展（3M 样本）============
defarray_pts_n(n,delta,theta_deg):
th=np.deg2rad(theta_deg)
k=int(round(n**0.5))
off=np.linspace(-(k-1)/2,(k-1)/2,k)*delta
xs,ys=np.meshgrid(off,off)
return(xs*np.cos(th)-ys*np.sin(th)).ravel(),(xs*np.sin(th)+ys*np.cos(th)).ravel()

N4=3_000_000
X4,Y4,Z4=sample_XYZ3(rng3,N4)
p1x=prob_any_hit(X4,Y4,Z4,*array_pts_n(1,dl_opt,th_opt),d_opt)
p33=prob_any_hit(X4,Y4,Z4,*array_pts_n(9,dl_opt,th_opt),d_opt)
p44=prob_any_hit(X4,Y4,Z4,*array_pts_n(16,dl_opt,th_opt),d_opt)

#============多弹相关性统计（36 对命中事件）============
defhit_matrix(X,Y,Z,px,py,d,chunk=1_000_000):
n=len(X);cnt=np.zeros((n,len(px)),dtype=np.int8)
forsinrange(0,n,chunk):
e=min(s+chunk,n)
Xs=X[s:e];Ys=Y[s:e];Zs=Z[s:e]
dx=np.abs(Xs[:,None]-px[None,:]);dy=np.abs(Ys[:,None]-py[None,:])
in_plane=(dx<=a)&(dy<=b)
h=np.sqrt(np.maximum(dx-a,0.0)**2+np.maximum(dy-b,0.0)**2)
v=np.maximum(Zs[:,None]-c-d,np.maximum(d-Zs[:,None]-c,0.0))
dist=np.sqrt(h**2+v**2)
hit=in_plane&(Zs[:,None]<=d+c)
hit|=in_plane&(Zs[:,None]>d+c)&((Zs[:,None]-c-d)<=R0)
hit|=(~in_plane)&(dist<=R0)
cnt[s:e]=hit.astype(np.int8)
returncnt

rngc=np.random.default_rng(11)
Nc=2_000_000
Xc,Yc,Zc=sample_XYZ3(rngc,Nc)
pxc,pyc=array_points(dl_opt,th_opt)
H=hit_matrix(Xc,Yc,Zc,pxc,pyc,d_opt).astype(np.float64)
C=np.corrcoef(H.T)
rs=[C[i,j]foriinrange(9)forjinrange(i+1,9)]
n_neg=sum(1forrinrsifr<0)
mean_r=float(np.mean(rs))
```

```
#============保存与输出============
res={
"table5_delta_sensitivity":[{"delta":t["delta"],"P":t["P"]}fortintbl5],
"table7_sigmaR":[{"param":t["param"],"value":t["value"],"P":t["P"]}fortintbl7],
"table8_sigz":[{"sigz":t["sigz"],"d_star":t["d_star"],"P_star":t["P_star"]}fortintbl8],
"table9_convergence":[{"N":t["N"],"freq":t["freq"],"dev":t["dev"],"se":t["se"]}fortintbl9],
"opt":{"d":d_opt,"delta":dl_opt,"theta_deg":th_opt},
"array_ext":{"single":float(p1x),"p33":float(p33),"p44":float(p44),
"p33_to_p44_gain":float((p44-p33)/p33)},
"correlation":{"pairs":36,"negative":n_neg,"mean_r":mean_r},
}
withopen(r"output/sensitivity_results.json","w",encoding="utf-8")asf:
json.dump(res,f,ensure_ascii=False,indent=2)

print("===表 5Δ灵敏度（θ=45°,d=%.2f）==="%d_opt)
fortintbl5:
print(f"Δ={t['delta']:.2f}:P={t['P']:.4f}")
print("===表 7σ/R 灵敏度===")
fortintbl7:
print(f"{t['param']}={t['value']}:P*={t['P']:.4f}")
print("===表 8σ_z 灵敏度===")
fortintbl8:
print(f"σ_z={t['sigz']}:d*={t['d_star']:.2f},P*={t['P_star']:.4f}")
print("===表 9 收敛性===")
fortintbl9:
print(f"N={t['N']:.0e}:freq={t['freq']:.4f},dev={t['dev']:.1e},se={t['se']:.1e}")
print(f"===4×4 扩展===单弹={p1x:.4f},3×3={p33:.4f},4×4={p44:.4f},增益={(p44-p33)/p33*100:.1f}%")
print(f"===相关性==={n_neg}/36 对负相关,平均 r={mean_r:.3f}")
print("savedoutput/sensitivity_results.json")
```